

TECHNICAL DOCUMENTATION

HealthChain

Patient-controlled, blockchain-audited health data access management system using a Sepolia smart contract, Laravel API, local PostgreSQL storage, and React wallet-based workflows.

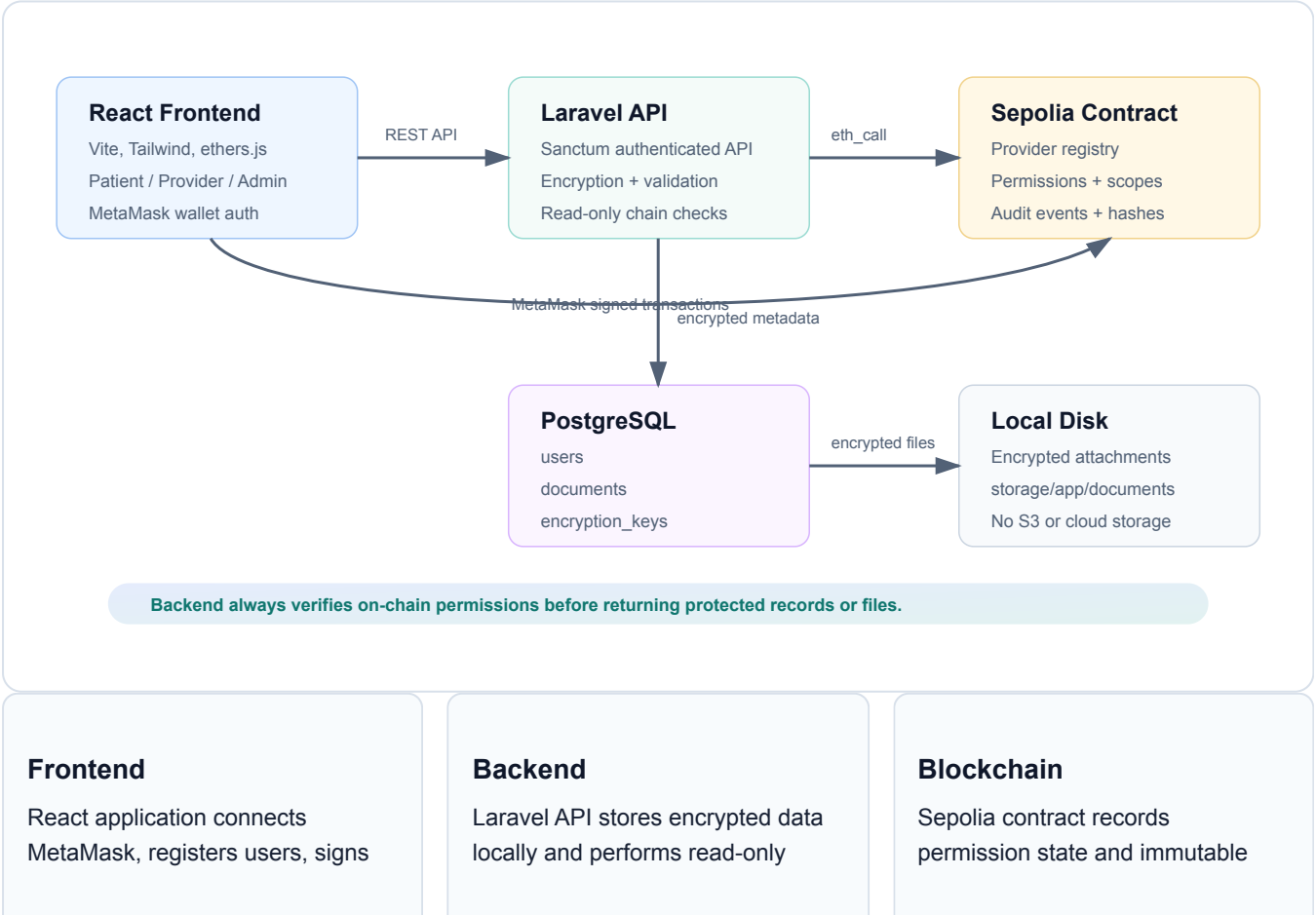
CONTRACT NETWORK Ethereum Sepolia, chainId 11155111	CONTRACT ADDRESS 0xc054E3bbCE68b074dbF23434082afc9F7aCFcbD2	LOCAL STACK React + Laravel 11 + PostgreSQL
---	---	---

1. Executive Summary

HealthChain separates access control from health record storage. Patients grant and revoke access on-chain, while sensitive medical profiles, documents, and encrypted file attachments remain in a local PostgreSQL-backed Laravel application. The smart contract stores only permission state, audit events, provider registry status, emergency contacts, data scopes, and SHA-256 anchors of plaintext records.

This design gives patients direct control over scope-specific provider access and gives administrators a verifiable audit trail without placing medical content on a public blockchain.

2. System Architecture



contract writes, and calls Laravel for off-chain data.

contract checks before serving protected data.

audit events without storing medical content.

3. Component Responsibilities

COMPONENT	TECHNOLOGY	PRIMARY RESPONSIBILITY
Smart Contract	Solidity 0.8.20, Hardhat	Provider registry, patient permissions, data scopes, emergency contacts, hash anchors, audit events.
Backend API	Laravel 11, Sanctum, PostgreSQL	User management, approval workflow, encrypted records, local file storage, permission enforcement.
Frontend	React 18, Vite, ethers.js v6, Tailwind	Wallet connection, dashboard workflows, user registration, signed smart contract writes, audit display.
Database	Local PostgreSQL	Stores users, encrypted document metadata, and encrypted patient data keys.
File Storage	Laravel local disk	Stores encrypted attachments under <code>storage/app/documents</code> .

4. Smart Contract Architecture

The `PatientHealthAccessControl` contract is the source of truth for provider registration and patient-granted access. It is intentionally small and stores no clinical content.

STATE	DESCRIPTION
<code>admin</code>	Super Admin wallet, set to the contract deployer.
<code>providers</code>	Maps provider wallet addresses to on-chain registration status and provider name.
<code>scopes</code>	Maps numeric scope IDs to sensitive health data categories.
<code>permissions</code>	Patient to provider to scope mapping with granted flag, expiry time, and grant timestamp.
<code>emergencyContacts</code>	Patient-controlled emergency contact address.
<code>dataHashes</code>	Patient and scope to SHA-256 plaintext record hash anchor.

Default Data Scopes

- 1 HIV STATUS
- 2 MENTAL HEALTH RECORDS
- 3 GENDER IDENTITY
- 4 GENERAL MEDICAL HISTORY
- 5 PRESCRIPTION RECORDS

Important Events

EVENT	PURPOSE
<code>ProviderRegistered</code>	Provider has been registered by the Super Admin on-chain.
<code>AccessGranted</code>	Patient granted time-limited provider access to a scope.
<code>AccessRevoked</code>	Patient revoked provider access.
<code>RecordAccessed</code>	Registered provider recorded successful scope access.
<code>AccessDenied</code>	Provider attempted access without valid permission.
<code>DataHashRegistered</code>	Patient anchored a plaintext record hash.

5. Backend Architecture

The Laravel API owns local persistence and data protection. It never writes to the blockchain. Contract write operations are signed by users through MetaMask in the frontend.

BlockchainService
Uses JSON-RPC read calls against Sepolia to verify provider status, permissions, and data hashes.

- `hasAccess(patient, provider, scopeId)`
- `isRegisteredProvider(address)`

EncryptionService
Manages per-patient data encryption keys and encrypts fields/files before local persistence.

- AES-256-GCM field encryption
- Per-patient DEKs

- `verifyHash(patient, scopeId, hash)`
- DEKs encrypted with Laravel `APP_KEY`

6. Database Schema

TABLE	PURPOSE	KEY COLUMNS
<code>users</code>	Patients, providers, and admins.	<code>wallet_address</code> , <code>role</code> , <code>is_approved</code> , <code>organisation</code> , <code>specialty</code>
<code>documents</code>	Encrypted document metadata and encrypted file references.	<code>patient_address</code> , <code>provider_address</code> , <code>scope_id</code> , <code>file_hash</code> , <code>tx_hash</code>
<code>encryption_keys</code>	Encrypted per-patient DEKs.	<code>patient_address</code> , <code>encrypted_key</code>
<code>personal_access_tokens</code>	Laravel Sanctum API authentication.	Token metadata and expiry fields.

7. Security and Access Control

1. **Backend authorization is mandatory.** The API verifies on-chain permission before returning documents or files.
2. **Frontend is not trusted.** The browser may request data, but Laravel independently verifies the permission state.
3. **Blockchain does not store medical data.** It stores permissions, audit events, scope names, and hashes only.
4. **Provider access requires two approvals.** Provider must be locally approved and registered on-chain.
5. **Files are encrypted before disk storage.** Attachments are stored under `storage/app/documents`.
6. **Hashes are computed from plaintext.** This allows integrity checking against the on-chain anchor.

Important: The current Sepolia RPC provider is Alchemy free tier. Its `eth_getLogs` endpoint limits block ranges and compute units. Admin audit logs are therefore paginated and loaded in small chunks.

8. Primary Workflows

Provider Approval

1. Provider registers in the frontend and is created as pending in PostgreSQL.
2. Admin reviews the provider profile.
3. Super Admin wallet signs `registerProvider(address, name)` in MetaMask.
4. Laravel marks the provider as approved locally.
5. Provider can access patient data only after patient grants scope permission.

Patient Grants Access

1. Patient connects MetaMask and selects provider, scopes, and expiry.
2. Frontend sends a signed contract transaction such as `grantAccess` or `grantAccessBatch`.
3. Contract stores permission state and emits `AccessGranted`.
4. Provider dashboard can discover accessible patient scopes.

Provider Reads or Adds Records

1. Provider calls Laravel API with patient, provider, and scope identifiers.
2. Laravel confirms provider is approved and registered on-chain.
3. Laravel calls `getPermission` and checks granted and expiry fields.
4. Only after successful verification does Laravel decrypt and serve metadata or file content.
5. When adding a document, Laravel encrypts fields and files before storage.

9. Audit Logging

Audit records are contract events fetched from Sepolia. Patient logs focus on events involving the patient wallet. Admin logs show a paginated system-wide view covering provider registration, scope setup, access grants, revocations, access events, emergency access, and hash registration.

Due to RPC limits, the application builds targeted log windows using known deployment blocks and permission timestamps. Admin audit pages load 15 records per page, reducing API pressure while preserving audit visibility.

10. Alchemy and RPC Provider Layer

HealthChain does not run its own Ethereum node. Instead, it uses Alchemy as the JSON-RPC gateway for Ethereum Sepolia. Alchemy receives RPC requests from the frontend and backend, forwards them to Sepolia infrastructure, and returns blockchain data such as contract state, transaction receipts, block metadata, and emitted events.

In this project, Alchemy is the only required external infrastructure service. The database, API, frontend, and file storage all run locally. Sepolia is accessed through the configured Alchemy endpoint: `https://eth-sepolia.g.alchemy.com/v2/YOUR_KEY`.

USAGE AREA	HOW HEALTHCHAIN USES ALCHEMY
Backend permission checks	Laravel uses Alchemy for read-only <code>eth_call</code> requests. Before serving protected documents, the backend calls the contract to verify provider registration and active patient permission.
Frontend contract reads	React uses an ethers.js <code>JsonRpcProvider</code> connected to Alchemy to read scopes, provider status, emergency contacts, permissions, and audit events.
Frontend contract writes	Write transactions are signed by MetaMask. Alchemy is used indirectly by MetaMask or the configured RPC endpoint to broadcast and confirm transactions on Sepolia.
Audit logs	The app calls <code>eth_getLogs</code> through Alchemy to retrieve contract events such as <code>AccessGranted</code> , <code>AccessRevoked</code> , <code>RecordAccessed</code> , and <code>ProviderRegistered</code> .
Block timestamps	After fetching logs, the app uses block reads through Alchemy to convert block numbers into human-readable event timestamps.

Why Pagination Is Required

Alchemy's free tier applies throughput and block-range limits to `eth_getLogs`. A broad request over tens of thousands of Sepolia blocks can be rejected with a compute-unit or block-range error. To avoid this, HealthChain does not request the entire contract history at once.

The admin audit screen uses paginated log loading. Each page returns 15 audit records and internally reads logs in small 10-block chunks. This keeps requests within Alchemy's free-tier constraints while still allowing the admin to browse the full audit trail page by page.

Operational recommendation: For production deployments, use a paid RPC plan, a dedicated Ethereum node, or an indexing service if audit volume becomes high. The current implementation is optimized for local development and Sepolia demonstration use.

11. Deployment and Runtime Configuration

AREA	REQUIRED SETTINGS
Contracts	<code>DEPLOYER_PRIVATE_KEY</code> , <code>SEPOLIA_RPC_URL</code>
Backend	<code>APP_KEY</code> , PostgreSQL credentials, <code>BLOCKCHAIN_RPC_URL</code> , <code>CONTRACT_ADDRESS</code> , <code>FILESYSTEM_DISK=local</code>
Frontend	<code>VITE_CONTRACT_ADDRESS</code> , <code>VITE_SEPOLIA_RPC_URL</code> , <code>VITE_CONTRACT_DEPLOY_BLOCK</code> , <code>VITE_API_URL</code>
Database	Local PostgreSQL database named <code>healthchain</code> .

12. Repository Structure

PATH	DESCRIPTION
<code>contracts/</code>	Hardhat project and Solidity contract.
<code>backend/</code>	Laravel API, migrations, models, controllers, services.
<code>frontend/</code>	React/Vite application and dashboards.
<code>docs/</code>	Generated project documentation.

Generated for HealthChain local development. Sensitive environment values, private keys, database passwords, RPC keys, and uploaded medical files must never be committed or distributed.